| Rank | Finding ID | Affected component / data flow | STRIDE category |
|---|---|---|---|
| 1 | F01 | DF10 P4→EE3 Tenant 1 API (via EE2, DF3, DS3 app1/app2 repos, P4) | Elevation of privilege |
| 2 | F02 | DS1 Redis rendered manifest cache; DF7 (P2, P3, P4) | Tampering |
| 2 | F03 | P2 Argo CD API Server (local admin; DS2 bootstrap Secret) | Spoofing |
| 2 | F04 | DS2 tenant cluster Secrets (also P9, DF8) | Information disclosure |
| 5 | F05 | DS1 Redis rendered manifest cache; DF7 | Information disclosure |
| 5 | F06 | EE1 Administrator workstation; DF1; P1 SSH bastion (DF2, DF14 as onward paths) | Spoofing |
| 7 | F07 | DF5 P3→P8 Git fetch (HTTP) | Tampering |
| 7 | F08 | DF3 EE2→P8 developer pushes (HTTP) | Spoofing |
| 9 | F09 | P4 Application Controller; DF10, DF11, DF12; EE3, EE4, EE5 | Tampering |
| 9 | F10 | DS2 Applications, AppProjects, RBAC and repo URL configuration | Tampering |
| 11 | F11 | P3 Repository Server; P4 Application Controller | Denial of service |
| 11 | F12 | DS4 Local logs; DF15 | Repudiation |
| 13 | F13 | EE4, EE5; DF11, DF12; DS3 (app3, app4 repos) | Tampering |
| 13 | F14 | DS3 Gitea repositories (app1, app3, app4); P8 | Information disclosure |

| | | | |
|---|---|---|---|
| 15 | F15 | P2 local admin account | Repudiation |
| 15 | F16 | DS3; P8 (commit history) | Repudiation |
| 15 | F17 | DF5 (app2 authenticated fetch) | Information disclosure |
| 18 | F18 | DS1 Redis; DF7 | Denial of service |
| 18 | F19 | P8 Gitea server; DF5 | Denial of service |
| 18 | F20 | P1 SSH bastion; DF1 | Denial of service |
| 21 | F21 | P1; DF14; DF2 | Repudiation |
| 21 | F22 | DF6 (P2, P4 → P3) | Spoofing |
| 21 | F23 | P5 ApplicationSet Controller; DF8 | Elevation of privilege |
| 24 | F24 | EE6 Notification email service; P6 | Spoofing |
| 24 | F25 | DF13 P6→EE6 (SMTP over TLS) | Information disclosure |
| 24 | F26 | P9 Ops Kubernetes API; DS2; all Ops components | Denial of service |

| Threat mechanism and consequence |
| --- |
| A developer pushes a manifest to main. With no independent approval and auto-sync, P4 applies it with the Tenant 1 cluster-wide credential. The manifest can reach the other app's namespace (TB8) or cluster-scoped objects, beyond the developer's own application. |
| Redis accepts unauthenticated, plaintext writes. An attacker overwrites cached rendered manifests so P4 applies attacker-chosen content to tenants, including Tenant 1, while Git stays clean and Git-based review never shows it. |
| Local admin is enabled, SSO is off, and the bootstrap password Secret remains. Whoever holds that password logs in as Argo admin and can create, change and sync apps across all tenants and alter Argo configuration. |
| Tenant 1–3 bearer tokens and TLS client configs, the app2 repo credential and the notification credentials sit in Kubernetes Secrets (base64 only, not presumed encrypted at rest). A reader can call tenant APIs directly, with Tenant 1 cluster-wide, outside Argo RBAC, AppProject limits and Argo-side logging. |
| Any client that reaches Redis can read cached rendered manifests, which can include Secret values or credentials embedded in manifests. The read crosses tenant lines because the cache holds every tenant's output. |
| A stolen or copied admin SSH key, or malware on the workstation, lets an attacker authenticate to the Internet-facing bastion with a single factor. From there they can use the tunnels to P2/P9 and the kubectl paths to tenant control planes. |
| Manifests are fetched over unauthenticated, unencrypted HTTP. An on-path or Gitea-impersonating host alters them or serves its own, and auto-sync deploys the result to tenants. |
| Developer credentials or tokens on plaintext HTTP can be captured and replayed. The attacker pushes to the developer's repo as them, and auto-sync deploys it. For app1/app2 this chains into F01. |
| Auto-prune and self-heal are disabled. Reverting a malicious commit does not remove resources it created (e.g., an extra RoleBinding or ServiceAccount), and out-of-band changes to live state persist uncorrected. |
| A malicious or compromised admin edits AppProject destinations, RBAC or an Application's repo URL, removing project restrictions or pointing deployment at an attacker-controlled repo that P3 fetches over anonymous HTTP. |
| One repo's oversized or pathological manifests exhaust the shared repo server and controller. Sync and rendering stall for all tenants; running workloads are unaffected. |
| Logs are local with no central immutable archive. An admin or other actor with access deletes them, or they rotate out under flooding. Evidence of other findings is lost and the actor can deny their actions. |
| A compromised developer pushes unreviewed malicious manifests that auto-sync into the app3 or app4 namespace. Namespace-scoped credentials (TB9, TB10) bound the damage. |
| Anonymous read lets any in-VPC client clone manifests and full commit history, enabling cross-tenant reconnaissance and possibly exposing embedded secrets. |

| |
|---|
| Admin actions are logged only as the single local 'admin' identity, so individual administrators can deny them. |
| Commits are unsigned, so author metadata can be forged. If force-push to main is allowed, history can be rewritten after Argo has synced. A malicious change is then attributed to someone else or erased. |
| The app2 repository credential crosses the network in cleartext HTTP. An observer captures it and reads the private app2 repo. |
| An unauthenticated client flushes or floods Redis, forcing re-rendering and load on P3. Sync and UI responsiveness degrade. |
| Anonymous clone or fetch floods from any in-VPC client stall pushes and Argo fetches, so new deployments stop. |
| Connection floods or exhaustion against the Internet-facing bastion block administrator access, including during an incident. |
| DF15 does not list the bastion or tenant API audit as log sources. Admin sessions and kubectl actions through the bastion on tenant clusters may be unattributable. |
| The package establishes TLS for gRPC, not caller authentication. A reachable client could pose as P2 or P4, request manifest generation and cause P3 to use its stored repo credential. |
| An actor able to create ApplicationSet custom resources in argocd gets Applications generated by the controller's own identity, bypassing P2 team RBAC review. No sets are active today. |
| Leaked notification credentials let an attacker send forged 'Argo' alerts, enabling phishing of admins or teams. |
| The package credits certificate validation for Kubernetes and gRPC, not SMTP. A downgrade or spoofed mail endpoint could capture the credentials and status content. |
| A single availability-zone failure takes Argo, Gitea, the bastion and the control plane down together. |

| Required initial attacker access | Additional assumption |
|---|---|
| Developer with write access to the app1 or app2 repo | AppProject restrictions (repository and destination selections only, per the package) are assumed not to constrain resource kinds or namespaces declared inside manifests. |
| Network reach to the Redis port: any Ops-cluster workload, or a compromised tenant workload | A tenant workload has a route to Redis (shared VPC/SG and no NetworkPolicy show connectivity is allowed, not that the route exists). Controllers use cached manifests without revalidating against P3. |
| Holder of the bootstrap password (leaked, retained, or read from the Secret) with network reach to P2 | The bootstrap password was never rotated and is still valid. |
| Holder of a leaked Argo Ops service-account token, or any identity with Secret read in argocd (including a malicious admin) | The reading identity actually has Secret read. Tenant tokens are long-lived and usable from the attacker's network position (e.g., via the bastion). |
| Same as F02 | Same reachability as F02. The cache actually contains sensitive values (the package says 'potentially'). |
| Anonymous external attacker holding the key or controlling EE1 | The key can be obtained from EE1. Usable Argo or kubectl credentials or sessions are reachable from EE1 or the bastion. |
| Compromised tenant workload or other in-VPC host | The attacker can intercept or redirect P3→P8 traffic (e.g., DNS or ARP manipulation). Connectivity alone doesn't give this. |
| On-path observer on the private path from the developer's network (TB2) | Credentials or tokens cross DF3 in a replayable form, and the observer can see the traffic. |
| Developer with write to one repo (Git path); compromised tenant workload (drift path) | None for the Git path. The drift path assumes the workload's service account can modify resources in its own namespace. |
| Malicious or compromised administrator | None |
| Developer with write to one repo | No manifest size, render-time or per-tenant resource limits are configured (the package is silent). |
| Malicious or compromised administrator | None |
| Developer with write to the app3 or app4 repo | None |
| In-VPC client, including a compromised tenant workload | The client can reach Gitea over HTTP. The repos contain sensitive configuration or secrets. |

| | |
|---|---|
| Malicious administrator | More than one administrator uses the same local account. |
| Developer with write to the repo | Force-push to main is permitted, and Gitea push audit is not retained beyond local DS4. |
| On-path observer in the VPC | The observer position as in F07. The credential is more than read-only if it also allows writes (unknown). |
| Same as F02 | Same reachability as F02. |
| In-VPC client (e.g., compromised tenant workload) | Reach to Gitea over HTTP. No rate limiting is configured. |
| Anonymous external attacker | None |
| Malicious administrator | No bastion session or tenant audit logging exists outside the model. |
| In-VPC or Ops-cluster workload | P3 does not require caller authentication beyond TLS, and the attacker has network reach. |
| Holder of Kubernetes create rights on custom resources in argocd (e.g., leaked Argo token) | The holder has such rights (not established). |
| Holder of leaked notification credentials | None |
| On-path attacker outside the VPC | P6 does not validate the mail server certificate. |
| None (environmental event) | None |

| Priority | Priority justification |
|---|---|
| **High** | Broadest credential in the system, a one-person gate, and no review window. Weaker if AppProject also constrains manifest contents. |
| **High** | Bypasses all Git controls, no authentication to defeat, and the blast radius includes cluster-wide Tenant 1. |
| **High** | Full Argo control with a single static credential, no second factor and no SSO. |
| **High** | One read yields credentials for all three tenants, including the cluster-wide one. |
| **Medium** | Easy, unauthenticated access, but impact depends on cache contents. |
| **Medium** | Only Internet entry and gateway to every admin path, but needs prior endpoint compromise and further credentials. |
| **Medium** | Strong effect (auto-sync, Tenant 1 reach) but needs an on-path position. |
| **Medium** | Gives repo write without a developer compromise, but needs an on-path position. |
| **Medium** | Cleanup after a bad commit leaves persistence, worst in cluster-wide Tenant 1. |
| **Medium** | Highest privilege level but a trusted role. The harm is weakening the controls that bound everyone else. |
| **Medium** | Cross-tenant impact from a single repo, but limited to the delivery plane. |
| **Medium** | Undermines attribution for every other finding. |
| **Low** | Developers are authorized to deploy there. The gap is only missing independent approval, with a namespace-bounded impact. |
| **Low** | Deliberate scenario setting; impact depends on repo contents. |

| | |
|---|---|
| **Low** | Depends on an unstated sharing practice. |
| **Low** | Plausible attribution gap, mitigated if Gitea records the authenticated pusher. |
| **Low** | Limited to one repo's read access on the stated facts. |
| **Low** | Cache loss is recoverable and the effect is temporary. |
| **Low** | Existing workloads keep running. |
| **Low** | Management plane only. Argo continues to sync. |
| **Low** | Depends on logging outside the model. |
| **Low** | Doubly conditional. |
| **Low** | Inactive component and needs significant prior access. |
| **Low** | Social-engineering vector with limited system impact. |
| **Low** | Requires an on-path position beyond the VPC. |
| **Low** | Not attacker-driven. Tenant workloads keep running. |

| Element ID | Type |
|---|---|
| EE1 | External entity |
| EE2 | External entity |
| EE3 | External entity |
| EE4 | External entity |
| EE5 | External entity |
| EE6 | External entity |
| P1 | Process |
| P2 | Process |
| P3 | Process |
| P4 | Process |
| P5 | Process |
| P6 | Process |
| P7 | Process |
| P8 | Process |
| P9 | Process |
| DS1 | Data store |
| DS2 | Data store |
| DS3 | Data store |
| DS4 | Data store |
| DF1 | Data flow |
| DF2 | Data flow |
| DF3 | Data flow |
| DF4 | Data flow |
| DF5 | Data flow |
| DF6 | Data flow |
| DF7 | Data flow |
| DF8 | Data flow |
| DF9 | Data flow |
| DF10 | Data flow |
| DF11 | Data flow |
| DF12 | Data flow |
| DF13 | Data flow |
| DF14 | Data flow |
| DF15 | Data flow |

| Name |
| --- |
| Administrator workstation |
| Application team developer |
| Tenant 1 Kubernetes API (app1, app2; cluster-wide credential) |
| Tenant 2 Kubernetes API (app3; namespace credential) |
| Tenant 3 Kubernetes API (app4; namespace credential) |
| Notification email service |
| SSH bastion |
| Argo CD API Server |
| Repository Server |
| Application Controller |
| ApplicationSet Controller |
| Notifications Controller |
| Dex (installed, SSO inactive) |
| Gitea server |
| Ops Kubernetes API |
| Redis rendered manifest cache |
| Ops Kubernetes persisted Secrets, ConfigMaps and custom resources |
| Gitea repositories and account data |
| Local Argo, Gitea and Kubernetes logs |
| EE1 to P1: SSH via Internet Gateway |
| P1 to P2, P9: tunneled HTTPS |
| EE2 to P8: developer Git pushes (HTTP) |
| P8 to DS3: local persistence |
| P3 to P8: Git fetch (HTTP) |
| P2, P4 to P3: TLS gRPC |
| P2, P3, P4 to DS1: Redis TCP, no TLS or auth |
| P2, P4, P5, P6 to P9: authenticated HTTPS |
| P9 to DS2: control plane storage |
| P4 to EE3: cluster-wide credential |
| P4 to EE4: namespace credential |
| P4 to EE5: namespace credential |
| P6 to EE6: SMTP with TLS |
| P1 to EE3, EE4, EE5: admin kubectl |
| Components to DS4: local log writes |

| Associated finding IDs | Finding count |
|---|---|
| F06 | 1 |
| F01, F08, F11, F13, F16 | 5 |
| F01, F04, F09 | 3 |
| F09, F13 | 2 |
| F09, F13 | 2 |
| F24, F25 | 2 |
| F06, F20, F21 | 3 |
| F03, F15 | 2 |
| F11, F22 | 2 |
| F01, F09, F11 | 3 |
| F23 | 1 |
| F24, F25 | 2 |
| No distinct threat identified | 0 |
| F14, F16, F19 | 3 |
| F04, F26 | 2 |
| F02, F05, F18 | 3 |
| F03, F04, F10, F24, F26 | 5 |
| F01, F13, F14, F16 | 4 |
| F12, F21 | 2 |
| F06, F20 | 2 |
| F06, F21 | 2 |
| F01, F08 | 2 |
| No distinct threat identified | 0 |
| F07, F17, F19 | 3 |
| F22 | 1 |
| F02, F05, F18 | 3 |
| F04, F23 | 2 |
| No distinct threat identified | 0 |
| F01, F09 | 2 |
| F09, F13 | 2 |
| F09, F13 | 2 |
| F25 | 1 |
| F06, F21 | 2 |
| F12, F21 | 2 |

## Argo CD GitOps Delivery — STRIDE Summary

*Counts are formulas over the Findings sheet. The coverage checklist is not part of the finding count.*

| Priority | Findings |
|---|---|
| High | 4 |
| Medium | 8 |
| Low | 14 |
| **Total** | **26** |

| STRIDE category | Findings | High | Medium | Low |
|---|---|---|---|---|
| Spoofing | 5 | 1 | 2 | 2 |
| Tampering | 5 | 1 | 3 | 1 |
| Repudiation | 4 | 0 | 1 | 3 |
| Information disclosure | 5 | 1 | 1 | 3 |
| Denial of service | 5 | 0 | 1 | 4 |
| Elevation of privilege | 2 | 1 | 0 | 1 |
| **Total** | **26** | **4** | **8** | **14** |